

Risk Parity from Scratch

Balancing a portfolio by risk rather than by capital — and why futures are the natural vehicle.

A traditional 60/40 portfolio looks balanced by *capital* — but not by *risk*. Because equities are $\sim 3\times$ as volatile as bonds, that 60/40 actually derives $\sim 90\%$ of its risk from stocks. **Risk parity** flips the question: instead of asking how much money each asset gets, it asks how much *risk* each contributes — and equalizes it. Futures are the natural vehicle, since risk parity leans on leverage to lift low-vol assets to their weight.

PROJECT CARD

CATEGORY

Portfolio Optimization

LIBRARIES

NumPy · Pandas · SciPy · Riskfolio-lib

ASSETS

ES, ZN, GC, HG, CL futures

TIMEFRAME

Jan 2015 – Dec 2024

USE

Balancing a multi-asset book by risk contribution rather than capital

1. Summary

Every asset in a portfolio contributes some share of the total risk — and those shares are rarely equal. Risk parity finds the weights that make each asset's **risk contribution** identical, so no single position dominates the portfolio's fate.

2. Intuition

2.1 Capital weight $\neq$ risk weight

Put 60% in stocks (15% vol) and 40% in bonds (5% vol). The stock sleeve's risk is $60\% \times 15\% = 9$ "units"; the bond sleeve's is $40\% \times 5\% = 2$. Stocks supply ~80–90% of the total. Your "balanced" portfolio is a stock portfolio wearing a bond costume.

2.2 Equalize the risk, not the dollars

Risk parity down-weights the volatile assets and up-weights the calm ones until each contributes the same risk. The

Because low-volatility assets (bonds) need large weights to pull their risk-weight up, risk parity portfolios are typically **levered** to reach an equity-like return — which is exactly why they're built with futures.

equity slice shrinks; bonds, gold and rates grow. The portfolio finally becomes diversified in the way that matters — by *source of risk*.

2.3 Leverage is a feature, not a bug

A risk-balanced portfolio is dominated by low-vol assets, so its raw volatility (and return) is low. To reach an equity-like target you scale the whole thing up with leverage — cheap and clean in futures. This is the mechanism behind "All Weather"-style funds.

3. Theory & Mechanics

For weights w and covariance Σ , portfolio volatility is $\sigma_p = \sqrt{w^T \Sigma w}$. Each asset's **marginal risk contribution** is:

$$MRC_i = \frac{\partial \sigma_p}{\partial w_i} = \frac{(\Sigma w)_i}{\sigma_p}, \quad RC_i = w_i \cdot MRC_i$$

and the RC_i sum exactly to σ_p (Euler's theorem). **Risk parity** seeks weights where all RC_i are equal:

$$RC_i = \frac{\sigma_p}{n} \quad \forall i$$

There's no closed form, so we minimize the dispersion of risk contributions numerically — first by hand in SciPy, then cross-checked with **Riskfolio-lib**.

4. Applied Example — Five Futures

4.1 A cross-asset futures universe

Five liquid futures spanning the major risk factors — equities, rates, and three commodities:

Ticker	Contract	Class
ES=F	S&P 500 E-mini	Equity
ZN=F	10y T-Note	Rates
GC=F	Gold	Metal (haven)
HG=F	Copper	Metal (cyclical)
CL=F	WTI Crude	Energy

```

TICKERS = ["ES=F", "ZN=F", "GC=F", "HG=F", "CL=F"]
NAMES = {"ES=F": "S&P 500", "ZN=F": "10y Note", "GC=F": "Gold", "HG=F": "Copper", "CL=F": "WTI"}
START, END = "2015-01-01", "2024-12-31"

def load_prices(tickers, start, end):
    """Continuous futures adjusted-close via yfinance."""
    import yfinance as yf
    df = yf.download(tickers, start=start, end=end, auto_adjust=True, progress=False)["Close"]
    return df[tickers].dropna()

px = load_prices(TICKERS, START, END)
rets = px.pct_change().dropna()

Sigma = rets.cov() * 252
vol = pd.Series(np.sqrt(np.diag(Sigma)), index=TICKERS)
print(f"{len(px)} trading days, {px.index[0].date()} → {px.index[-1].date()}\n")
print("Annualised volatility:")
print(vol.rename(index=NAMES).round(3))

```

Contract	Ann. volatility
S&P 500	17.9%
10y Note	5.5%
Gold	14.7%
Copper	22.1%
WTI	114.9%

DATA NOTE

Continuous =F series include roll gaps — and WTI's eye-watering 114.9% vol is dominated by the April 2020 episode, when the front-month contract briefly traded *negative* and daily percentage returns exploded. Fine for illustrating risk-parity mechanics (it makes the concentration problem vivid); production would use a back-adjusted series.

4.2 The problem with equal weight

Start naive: put 20% in each future. Watch how unequal the *risk* contributions are — crude alone supplies 87% of the portfolio's risk, while the 10y note contributes essentially nothing:

```

def risk_contributions(w, Sigma):
    """Risk contribution of each asset (sums to portfolio vol)."""
    w = np.asarray(w)
    sigma_p = np.sqrt(w @ Sigma @ w)
    mrc = (Sigma @ w) / sigma_p
    return w * mrc, sigma_p

n = len(TICKERS)
w_eq = np.repeat(1/n, n)
rc_eq, sig_eq = risk_contributions(w_eq, Sigma.values)

tbl = pd.DataFrame({"weight": w_eq, "risk contrib": rc_eq,
                    "risk share": rc_eq / sig_eq, index=[NAMES[t] for t in TICKERS]})
print(tbl.round(3))
print(f"\nPortfolio vol: {sig_eq:.1%} — but risk shares range "
      f"{(rc_eq/sig_eq).min():.0%} to {(rc_eq/sig_eq).max():.0%}, far from equal.")

```

Contract	Weight	Risk contrib	Risk share
S&P 500	20%	0.011	4.4%
10y Note	20%	-0.000	-0.0%
Gold	20%	0.006	2.4%
Copper	20%	0.016	6.3%
WTI	20%	0.219	86.9%

Portfolio vol is 25.2%, and the risk shares range from -0% to 87% — “equal weight” is a crude-oil bet in disguise. (The note’s share is slightly *negative*: its co-movement with the rest actually subtracts risk.)

4.3 Solve risk parity from scratch (SciPy)

Minimize the squared dispersion of risk contributions. At the optimum, every asset supplies exactly 1/n of the total risk.

```
from scipy.optimize import minimize

def rp_objective(w, Sigma):
    rc, sigma_p = risk_contributions(w, Sigma)
    target = sigma_p / len(w)
    return np.sum((rc - target) ** 2)          # equalise risk contributions

cons = ({ "type": "eq", "fun": lambda w: w.sum() - 1 },)
bnds = tuple((0.0, 1.0) for _ in range(n))
res = minimize(rp_objective, w_eq, args=(Sigma.values,), method="SLSQP",
               bounds=bnds, constraints=cons, tol=1e-12)

w_rp = res.x
rc_rp, sig_rp = risk_contributions(w_rp, Sigma.values)

tbl = pd.DataFrame({"RP weight": w_rp, "risk share": rc_rp / sig_rp},
                   index=[NAMES[t] for t in TICKERS])
print(tbl.round(3))
print(f"\nRisk shares now range {(rc_rp/sig_rp).min():.1%} to {(rc_rp/sig_rp).max():.1%} "
      f"— all ≈ {1/n:.0%}. Portfolio vol: {sig_rp:.1%}")
```

Contract	RP weight	Risk share
S&P 500	15.6%	20.0%
10y Note	55.2%	20.0%
Gold	15.7%	20.0%
Copper	11.1%	20.0%
WTI	2.4%	20.0%

All five risk shares land on exactly 20%. The allocation inverts the vol ranking: the calm 10y note gets 55% of the capital, wild WTI just 2.4% — and portfolio vol drops from 25.2% to 6.9%.

Figure 4.3 · Capital weight vs risk share — equal weight vs risk parity

Equal weighting gives every asset the same capital but very different risk shares; risk parity equalises them



Source: Author calculations, from the article's computed results.

4.4 Validate with Riskfolio-lib

Our hand-rolled solution should match the library's dedicated risk-parity optimizer to within solver tolerance — and it does, to 7.5×10^{-6} :

```
import riskfolio as rp

port = rp.Portfolio(returns=rets)
port.assets_stats(method_mu="hist", method_cov="hist")
w_lib = port.rp_optimization(model="Classic", rm="MV", rf=0, b=None)

compare = pd.DataFrame({
    "from scratch (SciPy)": w_rp,
    "Riskfolio-lib": w_lib["weights"].values,
}, index=[NAMES[t] for t in TICKERS])
compare["abs diff"] = (compare.iloc[:, 0] - compare.iloc[:, 1]).abs()
print(compare.round(4))
print(f"\nMax weight difference: {compare['abs diff'].max():.2e} (solver tolerance)")
```

Contract	From scratch (SciPy)	Riskfolio-lib
S&P 500	0.1557	0.1557
10y Note	0.5523	0.5523
Gold	0.1571	0.1571
Copper	0.1106	0.1106
WTI	0.0243	0.0243

4.5 Adding leverage to hit a volatility target

Risk parity's raw vol is low (bonds dominate). Scale the whole book to a target — say 10% annualised — with a single leverage multiplier. In futures this costs only margin, not capital.

```
target_vol = 0.10
leverage = target_vol / sig_rp
w_levered = w_rp * leverage

print(f"Unlevered RP vol: {sig_rp:.1%}")
print(f"Leverage to reach {target_vol:.0%}: {leverage:.2f}x")
```

```
print(f"Gross exposure: {w_levered.sum():.2f} (vs 1.00 unlevered)\n")
print(pd.Series(w_levered, index=[NAMES[t] for t in TICKERS], name="levered weight").round(3))
```

Contract	Unlevered weight	Levered weight
S&P 500	15.6%	22.6%
10y Note	55.2%	80.3%
Gold	15.7%	22.8%
Copper	11.1%	16.1%
WTI	2.4%	3.5%

Unlevered vol is 6.9%, so hitting 10% takes **1.45×** leverage — gross exposure 1.45 instead of 1.00. The note position alone is now 80% of capital: this is what “levered bonds” means in the risk-parity debate.

5. Conclusion

5a. Strengths

- **True diversification** — balances the *sources* of risk, not just the dollars
- **No return forecasts needed** — depends only on the covariance matrix, sidestepping MVO’s biggest weakness
- **Robust, stable weights** — small input changes barely move the allocation
- **Futures-native** — leverage to a vol target is cheap and clean in the futures market

5b. Weaknesses & Limitations

- **Leverage brings its own risks** — funding cost, margin calls, and forced deleveraging in a crisis (March 2020 hit risk-parity funds hard)
- **Covariance is still estimated** — correlations that break down in stress undermine the whole premise
- **Bond-heavy by construction** — vulnerable when rates and equities fall *together* (2022)

- **Ignores expected returns** — equalizing risk is agnostic about where return actually comes from

5c. Applications in Practice

- The engine behind “All Weather” / risk-parity funds (Bridgewater and imitators)
- A diversification overlay for multi-asset and CTA portfolios
- A return-forecast-free benchmark to test whether your alpha views actually add value

5d. Alternatives & Extensions

- **Hierarchical Risk Parity (HRP)** — clusters assets first, avoiding matrix inversion for stabler weights
- **Risk budgeting** — the general case, assigning *unequal* target risk shares by conviction
- **Mean-variance & Black-Litterman** — the return-driven alternatives (here and here)
- **CVaR risk parity** — equalize tail-risk contributions instead of variance contributions